FPGA based Hardware Scheduling approach for optimising DSP pipelines

Connor Sinclair

This thesis is presented as part of the requirements for the conferral of the degree:

Bachelor of Software Engineering (Hons.)

Supervisor:
Dr. Philip Leong

The University of Sydney
School of Engineering

2025

Declaration

I, *Connor Sinclair*, declare that this thesis is submitted in partial fulfilment of the requirements for the conferral of the degree *Bachelor of Software Engineering (Hons.)*, from the University of Sydney, is wholly my own work unless otherwise referenced or acknowledged. This document has not been submitted for qualifications at any other academic institution.

Connor Sinclair

October 8, 2025

Abstract

Field-programmable gate arrays (FPGAs) offer unprecedented parallelism and low latency for real-time audio processing. However, FPGA toolchains typically perform static compilation and mapping of fixed designs. In practice, audio signals and user-defined module graphs vary over time, presenting an opportunity to predictively reconfigure the FPGA fabric. In this thesis, I propose an operator-theoretic and state-space analysis framework that models DSP modules as dynamical systems, using observables and custom signal correlation metrics to forecast workload and adjust resource allocation. Unlike conventional flows, this approach can anticipate the evolving demands of a modular audio effect chain (including nonlinear modules) and remap kernels (e.g. heavy multipliers or trigonometric units) onto LUTs, DSP slices, or BRAMs in accordance to resource demands. Expected contributions include: (1) an analytical model of audio DSP modules grounded in operator theory (Koopman embeddings) and state-space representations; (2) coherent metrics of signal interaction for resource tuning; and (3) a prototype toolchain enabling runtime FPGA optimization in an accessible form. This work bridges a gap between control-theoretic modeling and FPGA architecture, offering audio engineers and embedded developers a more adaptive FPGA DSP platform that can improve throughput, support complex nonlinear & stochastic audio effects and enable massive real-time parallelisation.

Acknowledgements

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

1.2 Aims

1.3 Contributions

1.4 Thesis Structure

Chapter 2

Literature review

2.1 Dyadicization via Mahler (binomial) expansion on $\mathbb{Z}_2$

Theorem 1 (Mahler)**.** *For any continuous $f : \mathbb{Z}_2 \to \mathbb{Q}_2$, there exist unique coefficients $\{a_n\}_{n\geq 0} \subset \mathbb{Q}_2$ with $a_n \to 0$ in the 2-adic norm such that*

$$f(x) = \sum_{n=0}^{\infty} a_n \binom{x}{n}, \qquad \binom{x}{n} = \frac{x(x-1)\cdots(x-n+1)}{n!},$$

and the series converges uniformly on $\mathbb{Z}_2$. Moreover $a_n = \Delta^n f(0)$ where Δ is the forward difference operator.

Proof (sketch). See standard expositions in p-adic analysis. The polynomials $\binom{x}{n}$ form an orthonormal basis of $C(\mathbb{Z}_2, \mathbb{Q}_2)$ under the sup norm, and Newton–Gregory interpolation yields the coefficients a_n. Uniform convergence follows from $a_n \to 0$ in $\mathbb{Q}_2$. □

Corollary 1 (Dyadic evaluation and coefficient truncation)**.** *Fix $m \geq 0$ and evaluate the Mahler truncation*

$$S_M(x) := \sum_{n=0}^{M} a_n \binom{x}{n}$$

on the dyadic grid $x = k/2^m$ ($0 \leq k \leq 2^m$). Then each $\binom{k/2^m}{n}$ is a rational with denominator a power of 2; hence if the a_n are chosen in $\mathbb{Q}$ with denominators a power of 2, every value $S_M(k/2^m)$ is a dyadic rational. Further, the truncation error satisfies

$$\|f - S_M\|_{\infty,\mathbb{Z}_2} \leq \sup_{n>M} |a_n|_2,$$

so dyadic accuracy is controlled directly by the 2-adic decay of a_n.

Remarks for hardware. (i) The basis $\binom{x}{n}$ is *integer-valued* on $x \in \mathbb{Z}$, which helps cancellation of odd primes in arithmetic paths. (ii) If f is originally real-analytic on $[-1, 1]$, one may transfer to $\mathbb{Z}_2$ by encoding inputs on a dyadic grid and using the same truncated binomial basis; the resulting polynomial has dyadic coefficients after rounding a_n to a fixed 2^{-b} grid, with an a priori sup-norm bound inherited from the Mahler tail.

2.2 Walsh series: dyadic orthobasis and FPGA friendliness

Let $\{r_j\}_{j\geq 0}$ be the Rademacher functions on $[0,1)$, and define the Walsh functions (Paley ordering)

$$W_k(x) = \prod_{j=0}^{\infty} r_j(x)^{k_j}, \quad k = \sum_{j\geq 0} k_j 2^j,\ k_j \in \{0,1\}.$$

Then $\{W_k\}_{k\geq 0}$ is a complete orthonormal basis of $L^2([0,1))$. For $f \in L^2([0,1))$ its Walsh–Fourier series is

$$f(x) \sim \sum_{k=0}^{\infty} \widehat{f}(k)\, W_k(x), \qquad \widehat{f}(k) = \int_0^1 f(t)\, W_k(t)\, dt.$$

Dyadic rationality. If f is sampled on a dyadic grid of step 2^{-m} and the integrals are approximated by the exact rectangle rule on Walsh intervals, then each partial sum

$$S_N(x) = \sum_{k=0}^{N-1} \widehat{f}(k)\, W_k(x)$$

is a dyadic step function when the samples of f are dyadic rationals (the coefficients $\widehat{f}(k)$ become dyadic by construction). Computation uses the fast Walsh–Hadamard transform (FWHT), i.e. a butterfly network of adds/subtracts only.

Convergence and comparison to trigonometric series. Walsh series enjoy completeness and L^2 convergence analogous to Fourier series; uniform convergence holds under standard smoothness or bounded variation hypotheses adapted to dyadic intervals.

Chapter 3

Combined CSP & DSP approach to categorising signal families

3.1 Cyclostationary analysis

A signal with the property $x(t) = x(t+T)\ \forall\, t,\ T > 0$ is called *strictly periodic* (or *periodic pointwise*). A composition of periodic signals $h(t) = \sum_{i=0}^{N} x_i(t)$ is also periodic $\iff \frac{T_i}{T_j} \in \mathbb{Q}\ \forall\, i, j$ (i.e., the periods are commensurable). The period of $h(t)$ can therefore be described as

$$T = \mathrm{lcm}(T_0, T_1, \ldots, T_N).$$

A *wide-sense cyclostationary* (WSC) signal is defined by

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t+T)], \quad R_x(t,\tau) = R_x(t+T,\tau),$$

where $R_x(t,\tau) = \mathbb{E}[x(t)x^*(t+\tau)]$. That is, both the mean $\bar{x}(t)$ and the autocorrelation function $R_x(t,\tau)$ are periodic in t. More generally, a signal is cyclostationary *iff* there exists some nth-order nonlinear transformation of $x(t)$ that produces a finite additive sinusoidal component [].

The definition of the *spectral parameter* [] is

$$\hat{x}(\alpha) \ = \ \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} x(t)\, e^{-j2\pi\alpha t}\, dt.$$

For $x(t) = a\cos(2\pi\beta t + \theta)$:

$$x(t) = \tfrac{a}{2}\big(e^{j(2\pi\beta t+\theta)} + e^{-j(2\pi\beta t+\theta)}\big).$$

Substituting into the definition:

$$\hat{x}(\alpha) = \lim_{T\to\infty} \frac{a}{2T} \left[e^{j\theta} \int_{-T/2}^{T/2} e^{j2\pi(\beta-\alpha)t}\, dt \ + \ e^{-j\theta} \int_{-T/2}^{T/2} e^{-j2\pi(\beta+\alpha)t}\, dt \right].$$

Recalling that

$$\int_{-T/2}^{T/2} e^{j2\pi ft}\, dt = \frac{\sin(\pi fT)}{\pi f},$$

we obtain

$$\hat{x}(\alpha) = \frac{a}{2} \lim_{T\to\infty} \frac{1}{T} \left[e^{j\theta} \frac{\sin(\pi(\beta-\alpha)T)}{\pi(\beta-\alpha)} + e^{-j\theta} \frac{\sin(\pi(\beta+\alpha)T)}{\pi(\beta+\alpha)} \right].$$

Equivalently,

$$\hat{x}(\alpha) = \frac{a}{2} \lim_{T\to\infty} \left[e^{j\theta} \operatorname{sinc}((\beta-\alpha)T) + e^{-j\theta} \operatorname{sinc}((\beta+\alpha)T) \right],$$

where $\operatorname{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$.

If $\alpha = \beta$:

$$\hat{x}(\alpha) = \tfrac{a}{2} e^{j\theta}.$$

If $\alpha = -\beta$:

$$\hat{x}(\alpha) = \tfrac{a}{2} e^{-j\theta}.$$

Otherwise:

$$\hat{x}(\alpha) = 0.$$

Thus:

$$\hat{x}(\alpha) = \begin{cases} \dfrac{a}{2} e^{j\theta}, & \alpha = \beta, \\ \dfrac{a}{2} e^{-j\theta}, & \alpha = -\beta, \\ 0, & \text{otherwise.} \end{cases}$$

This demonstrates that the time-averaged inner product survives only when the test frequency α matches a sinusoidal component β of the signal.

3.1.1 Relation between Strict Periodicity and Cyclostationarity

Every strictly periodic signal is trivially wide-sense cyclostationary. Indeed, if $x(t) = x(t+T)$, then

$$\mathbb{E}[x(t)] = \mathbb{E}[x(t+T)], \quad R_x(t,\tau) = \mathbb{E}[x(t)x^*(t+\tau)] = \mathbb{E}[x(t+T)x^*(t+T+\tau)] = R_x(t+T,\tau).$$

Thus:

$$x(t) \text{ periodic} \implies x(t) \text{ WSC (cyclostationary)}.$$

The converse does not hold: cyclostationary signals need not be strictly periodic pointwise. For example, an amplitude-modulated process $x(t) = m(t)\cos(2\pi f_c t)$ with random data $m(t)$ can be cyclostationary with period $1/f_c$, while not being pointwise periodic.

3.2 Novel combined CSP, DSP approach to creating FPGA cache heirarchies

A function $f : A \to B$ is *foldable* iff $\exists\, A_0 \subseteq A$, $T = \{\tau_i : A \to A\}_{i\in I}$

such that

$$\forall x \in A,\, \exists\, \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

Where A_0 is called the *fundamental domain* & T, the set of transformations mapping from A_0 to A, the *recovery method*.

Motivating this definition with a classic example, consider $\sin(x)$ which has period 2π and can be represented with fundamental domain $\frac{\pi}{4}$ where the recovery method

$$T = \begin{cases} x, & 0 \leq x \leq \frac{\pi}{4}, \\ \pi - x, & \frac{\pi}{4} < x \leq \frac{\pi}{2}, \\ x - \pi, & \frac{\pi}{2} < x \leq \frac{3\pi}{2}, \\ 2\pi - x, & \frac{3\pi}{2} < x \leq 2\pi \end{cases}$$

$$f(x) = \begin{cases} f \circ \tau(x), & 0 \leq x \leq \pi, \\ -f \circ \tau(x), & \pi < x \leq 2\pi \end{cases}$$

Can be used to store effectively $4\times$ less memory M than the full period **OR** with the same memory footprint but $4\times$ greater granularity G. In general, the ratio between M & G, $\frac{M}{G}$, is called the *precision-memory gain tradeoff*. $M = \frac{P}{|A_0|}$, P is the period of the function.

Note: it is important that the recovery method $f \circ \tau(x)$ is relatively efficient / portable to an FPGA & that the tradeoff between the computational complexity of the recovery method & the memory gain M is not necessarily linear E.g. $\tau(x) \not\propto M$. A relevant example explored later on [] is the function $\arccos(x)$ which requires the computation of $\sqrt{x+1}$ on an FPGA in order to do the lookup on the reduced domain $\arccos : [0, \frac{1}{\sqrt{2}}] \to [-1, 1]$

3.2.1 Group-Theoretic Interpretation

The set of transformations T can be understood as the action of a *symmetry group* G on the domain A. In the sine example, the relevant group is the *dihedral group* D_4, which consists of rotations and reflections of a square [**armstrong1988groups**, **gallian2021contemporary**]. Each $\tau \in T$ corresponds to an element of D_4 acting on the domain of $\sin(x)$, with reflections accounting for sign changes and rotations corresponding to translations by $\pi/2$.

Thus, the concept of foldability is equivalent to requiring that the function f is invariant under the action of a group G, up to sign or phase factors. The *fundamental*

domain A_0 is then a representative region of A under this group action, and folding reduces storage by exploiting orbit representatives of G.

In group-theoretic language, the prior example is invariant under the group action up to a sign factor, i.e.,

$$\sin(x) = \epsilon(\tau)\,\sin(\tau(x)), \quad \epsilon(\tau) \in \{+1, -1\},\ \tau \in G.$$

The fundamental domain A_0 corresponds to a set of orbit representatives of G acting on A, and the recovery method reconstructs the full signal from this reduced domain.

3.2.2 Link to Cyclostationary Signals

Foldable functions are naturally related to cyclostationarity. A cyclostationary signal $x(t)$ has statistics that are periodic with period T.

For a foldable function like $\sin(x)$, the fundamental domain A_0 can be interpreted as a segment over which the first- and second-order statistics repeat under the action of G. In other words, folding exploits the same periodic structure that cyclostationary analysis detects: the mean and autocorrelation within A_0 are sufficient to reconstruct the signal statistics over the entire domain.

Thus, foldability can be seen as a deterministic, structural analogue of cyclostationarity: instead of averaging over time to detect periodic statistics, we use group-theoretic transformations to reconstruct the full signal from a reduced representative domain.

Let $f : A \to B$ be a foldable function with fundamental domain $A_0 \subseteq A$ and recovery transformations $T = \{\tau_i : A \to A\}_{i \in I}$ such that

$$\forall x \in A, \exists \tau \in T \text{ with } \tau(x) \in A_0 \quad \text{and} \quad f(x) = f \circ \tau(x).$$

This implies that the function f is invariant under the group action of G generated by T, and the entire domain A can be reconstructed from A_0 using the transformations in T.

Now, consider the autocorrelation function of f:

$$R_f(x, \Delta) = \mathbb{E}[f(x) f^*(x + \Delta)].$$

Since $f(x) = f(\tau(x))$ for some $\tau \in T$, we have

$$R_f(x, \Delta) = \mathbb{E}[f(\tau(x)) f^*(\tau(x + \Delta))].$$

Because $\tau(x), \tau(x + \Delta) \in A_0$ (up to another element of the group), all evaluations of $R_f(x, \Delta)$ are determined entirely by the behavior on A_0. Therefore,

$$R_f(x, \Delta) = R_f(\tau(x), \Delta), \quad \forall x \in A,\ \tau \in T,\ \tau(x) \in A_0.$$

Thus, the fundamental domain A_0 captures all statistical information of the function f, analogous to how the fundamental domain in cyclostationary signal processing captures the periodic statistical properties of a signal.

3.2.3 Procedure for storing arbitrary signals

A procedure to *caching* an arbitrary signal $x(t)$ is the following:

1. Decompose the signal into a series of harmonics (for analysis purposes). In practice, this can be approximated via autocorrelation or spectral analysis rather than explicit FFT computation.

2. Use autocorrelation (ACF) analysis on the fundamental harmonic to estimate the period of a noisy signal within a specified SNR tolerance.

3. Iteratively refine the period estimate using heuristics, e.g., minimize spectral leakage if the period is undershot, or adjust lags using peak prominence methods.

4. Determine if the signal is *foldable* by checking whether there exists a fundamental domain A_0 and a set of transformations T such that

$$\forall x \in A,\ \exists \tau \in T \text{ with } \tau(x) \in A_0 \text{ and } f(x) = f \circ \tau(x).$$

   This step is limited to signals with known symmetries (e.g., trigonometric functions, piecewise monotonic functions).

5. Construct a recovery method that maps any $x \in A$ to A_0 via the transformations in T, taking into account the desired precision-memory gain tradeoff based on control-theoretic or resource-demand considerations.

6. Implement the recovery method in a synthesizable and efficient form for FPGA deployment, ensuring that the computational complexity of applying $\tau(x)$ does not outweigh the memory savings.

Methods for *(1), (2) and (3)*, previously discussed and implemented, are the subject of extensive optimization effort. Steps *(4), (5), and (6)*, however, are considerably more difficult to achieve in practice on a completely unrestricted problem domain. Indeed, this paper focuses on a subset of signal definitions that can be reliably categorized, as well as on certain elementary signals that are relatively commonplace and often appear as components within more stochastic processes.

3.3 Synthesizable construction of the recovery method, T, for FPGA's

3.3.1 Efficient FPGA Implementation of $\arccos(x)$ Using 'Quarter'-Domain Folding

In previous sections, we discussed foldable functions and their use in memory-efficient lookup. In this section, we consider the case of $\arccos(x)$, which initially seems trivial but actually presents valuable insights for efficient computation on hardware.

Fundamental Domain Recovery The key identity used for recovery from a quarter-domain representation is:

$$\tau(x) = \frac{1}{\sqrt{2}}\sqrt{x+1},$$
$$f(x) = 2 \cdot \arccos(\tau(x)) \tag{3.1}$$
$$\implies \arccos(x) = 2 \arccos\left(\frac{1}{\sqrt{2}}\sqrt{x+1}\right). \tag{3.2}$$

Applying a third-order Taylor series expansion to $\sqrt{x+1}$ yields:

$$16\sqrt{x+1} \approx x^3 - 2x^2 + 8x + 16.$$

Unlike the arcsin third-order polynomial, this polynomial can be factored over a finite field:

$$(x+1)(x^2+x+1).$$

To recover the original polynomial, we add the remaining series:

$$T_2 = -4x^2 + 7x + 15.$$

Efficient Hardware Computation Computing the first term:

$$T_1 = (x+1)(x^2+x+1)$$

(**Note** that the second term T_2 can be computed using precomputed values)

$T_2 = -4x^2+7x+15 = -4x^2+8x+16-(x+1) = -(x^2 << 2)+((x+1)+1) << 3)-(x+1)$

Which can be implemented as follows:

```
// Begin computation of tau(x) for arccos(x)
// arccos in [0, 1/sqrt(2)], Q2.21 format (24 bit data-width)

// Clock cycle 1: compute shared terms
t1 = x + 1;
x_squared = x * x;

// Clock cycle 2: compute T1
t1 = t1 * (x_squared + t1);

// Clock cycle 2: concurrent with T1, compute T2
t2 = -(x_squared << 2) + ((t1 + 1) << 3) - t1;

// Clock cycle 3: add finite-field-factorization correction term, T2, to T1
tau = t1 + t2;

// Clock cycle 4: right-shift by 4 to get sqrt(x + 1), multiply by 1/sqrt(2) term
localparam inv_sqrt2_q = ...; // 1/sqrt(2) stored in Q2.21 format
tau = (tau >> 4) * inv_sqrt2_q;
```

```
// Do the other required computations to get T

// Clock cycle 5/6: quantization of tau(x) to indice present in BRAM, LUT lookup
logic [23:0] T;
acos_tau_quantize f (.tau(tau), .f(T), .clk(clk));

// Clock cycle 7: final result
T = T << 1;
```

Resource Analysis and Benefits The total cost for this third-order implementation of $\tau(x)$:

- 3 multiplications (1 for shared term, 1 for T_1, 1 for multiplication by $\frac{1}{\sqrt{2}}$)
- 6 additions/subtractions (1 for shared term, 1 for T_1, 3 for T_2, 1 for the final sum)
- 4 bit-shifts and 1 bitwise inversion (sign flip)

Advantages:

1. No transcendental operations in the argument apart from the $\frac{1}{\sqrt{2}}$ multiplier, simplifying quantization.
   - Quantization step can be bypassed to save a clock cycle
   - Achieved via computing the arccos lut values along a non-linear axis I.e. according to the distribution $\tau(x)$
2. Only a single DSP unit is required for all multiplies if the data width allows single-cycle multiplication.
3. No additional storage for constants required except for $\frac{1}{\sqrt{2}}$.
4. The computation is easily pipelined.
5. Bit-growth is predictable, scaling relative to $\deg(\tau(x))$

3.3.2 Generalizing recovery method construction technique

General procedure to obtain synthesizable FPGA code, given a known T

1. For some $\tau(x)$, express $\tau(x)$ with dyadic rational coefficients
2. Find a recursive finite factorization of $\mathbb{F}(\tau(x))$
3. Find and isolate shared/successive terms in $\mathbb{F}(\tau(x))$ [**reference˙needed**]
4. Pipeline and synthesize for target FPGA

3.3.3 Setup: Walsh basis and discrete (dyadic) synthesis

Let $\{W_k\}_{k\geq 0}$ denote the Walsh–Paley system on $[0,1)$, an orthonormal basis of $L^2([0,1))$ [**Paley1932**, **Fine1949**, **GES1991**]. For any $f \in L^2([0,1))$, its Walsh expansion is

$$f(x) \sim \sum_{k=0}^{\infty} \widehat{f}(k)\, W_k(x), \qquad \widehat{f}(k) := \int_0^1 f(t)\, W_k(t)\, dt.$$

Fix an integer $m \geq 1$ and set $N := 2^m$. Let the dyadic grid be

$$x_j := \frac{j}{N}, \quad j = 0, 1, \ldots, N-1.$$

Define the (vector of) samples $y_j := f(x_j)$. The *discrete* Walsh–Hadamard synthesis uses the $N \times N$ Hadamard matrix H_N with entries $(H_N)_{k,j} = W_k(x_j) \in \{\pm 1\}$ (Paley ordering). Then the length-N coefficient vector $c = (c_k)_{k=0}^{N-1}$ is

$$c = \frac{1}{N} H_N\, y, \qquad \text{i.e. } c_k = \frac{1}{N} \sum_{j=0}^{N-1} y_j\, W_k(x_j).$$

The length-N partial Walsh sum is the step function

$$S_N(x) := \sum_{k=0}^{N-1} c_k\, W_k(x),$$

which equals the conditional expectation of f onto the dyadic σ-algebra at scale 2^{-m} (the average of f on each dyadic subinterval).

3.3.4 Forcing dyadic coefficients via dyadic sampling & quantization

Step 1 (Dyadic quantizer). Fix $b \in \mathbb{N}$. Define the dyadic quantizer $Q_b : \mathbb{R} \to 2^{-b}\mathbb{Z}$ by

$$Q_b(u) := 2^{-b}\, \mathrm{round}(2^b u).$$

Apply Q_b to the samples:

$$\tilde{y}_j := Q_b\big(f(x_j)\big) \in 2^{-b}\mathbb{Z}.$$

Step 2 (FWHT / discrete coefficients). Compute

$$\tilde{c}_k := \frac{1}{N} \sum_{j=0}^{N-1} \tilde{y}_j\, W_k(x_j).$$

Since each $\tilde{y}_j$ is a dyadic rational and $W_k(x_j) \in \{\pm 1\}$, we have

$$\tilde{c}_k \in 2^{-(b+m)}\mathbb{Z} \quad \Rightarrow \quad \tilde{c}_k \text{ is dyadic}, \quad \forall k.$$

Thus the *quantized discrete Walsh expansion*

$$\tilde{S}_N(x) := \sum_{k=0}^{N-1} \tilde{c}_k\, W_k(x)$$

has *dyadic* coefficients and is piecewise constant on dyadic intervals.

3.3.5 Worked example: $f(x) = \sin(2\pi x)$

Take $f(x) = \sin(2\pi x)$, which has the non-dyadic Fourier series $f(x) = \frac{1}{2i}\big(e^{2\pi ix} - e^{-2\pi ix}\big)$ (coefficients are not dyadic). Choose m, b and define

$$x_j = \frac{j}{2^m}, \qquad \tilde{y}_j = Q_b\big(\sin(2\pi x_j)\big).$$

Compute Walsh coefficients

$$\tilde{c}_k = 2^{-m} \sum_{j=0}^{2^m-1} \tilde{y}_j \, W_k\Big(\frac{j}{2^m}\Big) \in 2^{-(b+m)}\mathbb{Z}.$$

Hence the Walsh approximation $\tilde{S}_{2^m}(x) = \sum_{k=0}^{2^m-1} \tilde{c}_k W_k(x)$ is a *dyadic series* that replaces the non-dyadic trigonometric series.

Error decomposition and guarantees

Write three errors:

$$\|f - \tilde{S}_N\|_{L^2} \le \underbrace{\|f - \mathbb{E}[f \,|\, \mathcal{D}_m]\|_{L^2}}_{\text{dyadic averaging error}} + \underbrace{\|\mathbb{E}[f \,|\, \mathcal{D}_m] - \mathbb{E}[\tilde{f} \,|\, \mathcal{D}_m]\|_{L^2}}_{\text{sample quantization}} + \underbrace{\|\mathbb{E}[\tilde{f} \,|\, \mathcal{D}_m] - \tilde{S}_N\|_{L^2}}_{\text{discrete vs. continuous Walsh}},$$

where $\mathcal{D}_m$ is the dyadic σ-algebra at scale 2^{-m} and $\tilde{f}$ is the function matching the quantized samples on grid points and extended piecewise constantly on each dyadic interval.

- (*Averaging error*) $\mathbb{E}[f \,|\, \mathcal{D}_m]$ is the best L^2-approximation of f by functions constant on dyadic intervals. As $m \to \infty$, $\|f - \mathbb{E}[f \,|\, \mathcal{D}_m]\|_{L^2} \to 0$ (martingale convergence; see [**GES1991**]). If f is Lipschitz with constant L, then

  $$\|f - \mathbb{E}[f \,|\, \mathcal{D}_m]\|_{L^\infty} \le L\, 2^{-m}.$$

- (*Quantization error*) Since $|\tilde{y}_j - y_j| \le 2^{-b-1}$,

  $$\|\mathbb{E}[f \,|\, \mathcal{D}_m] - \mathbb{E}[\tilde{f} \,|\, \mathcal{D}_m]\|_{L^2} \;\le\; 2^{-b-1}.$$

- (*Discrete/continuous Walsh match*) For $N = 2^m$, the rectangle rule over dyadic intervals is *exact* for Walsh atoms W_k (they are constant on each interval), hence

  $$\mathbb{E}[\tilde{f} \,|\, \mathcal{D}_m](x) = \sum_{k=0}^{N-1} \tilde{c}_k \, W_k(x) = \tilde{S}_N(x),$$

  so this term vanishes.

Therefore, with $N = 2^m$,

$$\boxed{\|f - \tilde{S}_N\|_{L^2} \;\le\; \|f - \mathbb{E}[f \,|\, \mathcal{D}_m]\|_{L^2} \;+\; 2^{-b-1},}$$

and every coefficient $\tilde{c}_k$ is dyadic: $\tilde{c}_k \in 2^{-(b+m)}\mathbb{Z}$.

Chapter 4

Architectural Design

4.1 Procedure for approximating Error statistics on filter kernels

One of the fundamental results in DSP is that the ideal window function

$$w(t) = \text{sinc}(t)$$

in the time domain corresponds to a perfect rectangular pulse in the frequency domain [**OppenheimDSP**]. However, as the envelope $\frac{1}{x}$ decays the amplitude of $\sin(x)$ monotonically, the function has an infinite domain that is impossible to capture discretely. Thus, there is an associated truncation error $T(x)$, namely, it is the error between the tail & partial power series. This implies the existence of an associated error term $\mathbb{E}(x)$ that depends on the truncation point x. The second contributor is *coefficient quantization error*.

4.1.1 Optimal Coefficient Quantization via Projection Methods

Consider a function $f(x)$ represented as a polynomial series:

$$f(x) = \sum_{n=0}^{N} c_n x^{2n} \tag{4.1}$$

where c_n are the series coefficients. In fixed-point hardware implementations, these coefficients must be quantized to dyadic rationals of the form $\tilde{c}_n = a_n/2^{m_n}$, where $a_n \in \mathbb{Z}$ and m_n determines the bit precision allocated to coefficient n.

The quantization error for coefficient n is:

$$\epsilon_n = c_n - \tilde{c}_n \tag{4.2}$$

The total approximation error introduced by coefficient quantization is:

$$E(x) = f(x) - \tilde{f}(x) = \sum_{n=0}^{N} \epsilon_n x^{2n} \tag{4.3}$$

Our objective is to determine the optimal bit allocation vector $\mathbf{m} = [m_0, m_1, \ldots, m_N]$ that minimizes the projected error under a total bit budget constraint $\sum_{n=0}^{N} m_n \leq B_{\text{total}}$ for a LPF. Framing the problem in terms of an analytically continued series, $R(x)$, exposes a singular metric that packs in both 'phase' and 'bit' quantization error, expressed as a sensitivity matrix. This is effective on the computational front, allowing for a signal to be dually low-passed & re-weighted according to tolerance error. As will be shown, it also a neccessery ingredient for autonomously reducing memory pressure via. a parameterized 'sacrifice on accuracy'.

4.1.2 Projection Operator and Error Metric

Rather than minimizing the error in the standard L^2 norm, we introduce a projection operator $R(x)$ that captures the application-specific error weighting. The projected error is defined as:

$$\mathcal{E}_{\text{proj}} = \langle E(x), R(x) \rangle = \int_0^{x_{\max}} E(x) \cdot \overline{R(x)}\, dx \tag{4.4}$$

where $\overline{R(x)}$ denotes the complex conjugate of $R(x)$, and $\langle \cdot, \cdot \rangle$ represents the inner product in $L^2[0, x_{\max}]$.

Substituting the error polynomial:

$$\mathcal{E}_{\text{proj}} = \int_0^{x_{\max}} \left[\sum_{n=0}^{N} \epsilon_n x^{2n} \right] \overline{R(x)}\, dx = \sum_{n=0}^{N} \epsilon_n \int_0^{x_{\max}} x^{2n} \overline{R(x)}\, dx \tag{4.5}$$

4.1.3 Sensitivity Matrix Construction

The sensitivity of the projected error to coefficient n is defined as:

$$S_n = \left| \frac{\partial \mathcal{E}_{\text{proj}}}{\partial \epsilon_n} \right| = \left| \int_0^{x_{\max}} x^{2n} \overline{R(x)}\, dx \right| \tag{4.6}$$

This represents how strongly a unit error in coefficient c_n affects the overall projected error. For a complex projector $R(x) = R_{\text{real}}(x) + iR_{\text{imag}}(x)$, we can decompose the sensitivity into real and imaginary components:

$$S_n^{\text{real}} = \left| \int_0^{x_{\max}} x^{2n} R_{\text{real}}(x)\, dx \right|, \quad S_n^{\text{imag}} = \left| \int_0^{x_{\max}} x^{2n} R_{\text{imag}}(x)\, dx \right| \tag{4.7}$$

The *phase sensitivity* and *magnitude sensitivity* can be defined as:

$$S_n^{\text{phase}} = S_n^{\text{imag}} \tag{4.8}$$

$$S_n^{\text{mag}} = S_n^{\text{real}} \tag{4.9}$$

The complete sensitivity matrix is:

$$\mathbf{S} = \begin{bmatrix} S_0^{\text{mag}} & S_0^{\text{phase}} \\ S_1^{\text{mag}} & S_1^{\text{phase}} \\ \vdots & \vdots \\ S_N^{\text{mag}} & S_N^{\text{phase}} \end{bmatrix} \tag{4.10}$$

4.1.4 Quantization Error Bounds

For dyadic quantization with m_n bits allocated to coefficient n, the quantization error is bounded by:

$$|\epsilon_n| \leq \frac{1}{2^{m_n+1}} \tag{4.11}$$

The worst-case projected error is therefore:

$$|\mathcal{E}_{\text{proj}}| \leq \sum_{n=0}^{N} S_n |\epsilon_n| \leq \sum_{n=0}^{N} \frac{S_n}{2^{m_n+1}} \tag{4.12}$$

4.1.5 Optimal Bit Allocation Strategy

Method 1: Sensitivity-Proportional Allocation

Allocate bits proportionally to sensitivity:

$$m_n = m_{\text{base}} + \left\lfloor \alpha \cdot \frac{S_n}{S_{\max}} \right\rfloor \tag{4.13}$$

where $S_{\max} = \max_n S_n$, m_{base} is the minimum bit allocation, and α controls the dynamic range of bit allocation (typically $\alpha \in [3, 8]$).

The allocation is subject to the constraint:

$$\sum_{n=0}^{N} m_n = B_{\text{total}} \tag{4.14}$$

Method 2: Weighted Phase-Magnitude Allocation

For applications requiring different emphasis on phase versus magnitude accuracy, define a weighted sensitivity:

$$S_n^{\text{weighted}} = \sqrt{w_{\text{mag}} (S_n^{\text{mag}})^2 + w_{\text{phase}} (S_n^{\text{phase}})^2} \tag{4.15}$$

where w_{mag} and w_{phase} are application-dependent weights satisfying $w_{\text{mag}} + w_{\text{phase}} = 1$.

Method 3: Greedy Optimization

For the global optimum under constraint (4.12), we employ a greedy algorithm. The algorithm proceeds as follows:

Greedy Bit Allocation Algorithm:

1. Initialize $m_n = 1$ for all $n \in \{0, \ldots, N\}$
2. Set $B_{\text{remaining}} = B_{\text{total}} - (N + 1)$
3. While $B_{\text{remaining}} > 0$:

(a) Compute the error reduction for adding one bit to coefficient n:

$$\Delta_n = S_n \left(\frac{1}{2^{m_n+1}} - \frac{1}{2^{m_n+2}} \right) \quad \text{for all } n \tag{4.16}$$

(b) Find the coefficient with maximum error reduction: $n^* = \arg\max_n \Delta_n$

(c) Increment the bit allocation: $m_{n^*} \leftarrow m_{n^*} + 1$

(d) Decrement the remaining budget: $B_{\text{remaining}} \leftarrow B_{\text{remaining}} - 1$

4. Return the optimal bit allocation vector $\mathbf{m} = [m_0, m_1, \ldots, m_N]$

This greedy approach is optimal for the additive error model in (4.12) and has computational complexity $O(B_{\text{total}} \cdot N)$.

4.1.6 Evaluation of Projected Error

After bit allocation, the actual projected error can be computed as:

$$\mathcal{E}_{\text{proj}} = \left| \sum_{n=0}^{N} (c_n - \tilde{c}_n) \int_0^{x_{\max}} x^{2n} \overline{R(x)}\, dx \right| \tag{4.17}$$

For numerical evaluation, discretize the integral:

$$\mathcal{E}_{\text{proj}} \approx \left| \sum_{n=0}^{N} \epsilon_n \sum_{k=0}^{K-1} x_k^{2n} \overline{R(x_k)} \Delta x \right| \tag{4.18}$$

where $x_k = k \cdot x_{\max}/K$ and $\Delta x = x_{\max}/K$.

The real and imaginary components of the error are:

$$\mathcal{E}_{\text{real}} = \text{Re} \left[\sum_{n=0}^{N} \epsilon_n \int_0^{x_{\max}} x^{2n} \overline{R(x)}\, dx \right] \tag{4.19}$$

$$\mathcal{E}_{\text{imag}} = \text{Im} \left[\sum_{n=0}^{N} \epsilon_n \int_0^{x_{\max}} x^{2n} \overline{R(x)}\, dx \right] \tag{4.20}$$

4.1.7 Projector Derivation for Sinc Function

The power series for $\mathrm{sinc}(x)$ is given as:

$$\mathrm{sinc}(x) = \sum_{n=0}^{\infty} \frac{-1^n}{\Gamma(2n)} x^{2n+1}$$

But due to the necessity of considering truncated power series, introduce the upper bound N.Then re-indexing to $2N$ we obtain:

$$\mathrm{sinc}(x) = \sum_{k=0}^{2N} \frac{(-1)^{\lfloor k/2 \rfloor} x^{k+1}}{\Gamma(k)} \quad = \sum_{k=0}^{2N} \delta_{k \equiv 0 \,(\mathrm{mod}\, 2)} \cdot \frac{i^k \cdot x^{k+1}}{\Gamma(k)}$$

Implicitly allowing for the analytic continuation of E ($E : \mathbb{R} \to \mathbb{R} \to E : \mathbb{C} \to \mathbb{C}$) due to the introduction of the *Kronecker delta*[].This projector naturally selects even-indexed coefficients, which are precisely the non-zero terms in the sinc Taylor series.

Using the fact that we can exchange the sum & integrand due to linearity[] and using the contour representation of the kroenecker delta for term extraction[] (I.e. as a *cauchy kernel*) we derive:

$$\frac{1}{2\pi i} \oint_{\gamma} \sum_{k=0}^{2N} \frac{1}{\zeta(1-\zeta)^2} \frac{i^k \cdot x^{k+1}}{\Gamma(k)} d\zeta$$

(**??**)

Then let

$$\begin{aligned} S &:= \sum_{k=0}^{2N} \frac{i^k \cdot x^{k+1}}{\Gamma(k)} \\ M &:= 2N \\ \implies S &= -i \sum_{j=1}^{M+1} \frac{(ix)^j}{\Gamma(j)} \\ &= -i \left(\exp(ix) - 1 - \sum_{j=M+2}^{\infty} \frac{(ix)^j}{\Gamma(j)} \right) \end{aligned}$$

(**??**)

We may then use the incomplete-gamma identity[] for the power series of e^x yielding:

$$S = -i \left[\exp(ix) \left(\frac{\Gamma(n+2, ix)}{\Gamma(n+2)} - 1 \right) \right] \quad = i - \frac{i \, \exp(ix) \, \Gamma(n+2, ix)}{\Gamma(n+2)}$$

(**??**)

The projector identity*[] finally gives:

$$\begin{aligned} \mathbb{P}(x) &= \frac{1}{2\pi i} \oint_{\gamma} \mathrm{B}(z) \mathrm{A}(\frac{x}{z}) dz \\ \implies R(x) &= \frac{1}{2\pi \Gamma(M+2)} \oint_{\gamma} \frac{\Gamma(M+2) - \exp(\frac{ix}{z}) \Gamma(M+2, \frac{ix}{z})}{z(1-z)^2} dz \end{aligned}$$

(??)
Which can be expressed more robustly using a change of variable $\frac{x}{z} \to \theta$

$$\frac{1}{2\pi\Gamma(K)} \oint_{C_\theta} (\Gamma(K) - e^{i\theta}\Gamma(K, i\theta))(\frac{\theta}{\theta^2 - x^2})d\theta$$

(??)
Where C is the unit image circle.

By the residue theorem, evaluating at the poles $\theta = \pm x$:

$$\frac{1}{2\pi\Gamma(K)} \oint_{C_\theta} \frac{\theta\Gamma(K)}{\theta^2 - x^2} d\theta = \frac{1}{2\Gamma(K)} \cdot 2\pi i \left[\text{Res}_{\theta=x} + \text{Res}_{\theta=-x}\right] \Gamma(K)$$

The residues are:

$$\text{Res}_{\theta=x} \frac{\theta}{\theta^2 - x^2} = \lim_{\theta \to x} \frac{\theta(\theta - x)}{(\theta - x)(\theta + x)} = \frac{x}{2x} = \frac{1}{2}$$
$$\text{Res}_{\theta=-x} \frac{\theta}{\theta^2 - x^2} = \lim_{\theta \to -x} \frac{\theta(\theta + x)}{(\theta - x)(\theta + x)} = \frac{-x}{-2x} = \frac{1}{2}$$

Therefore:

$$\frac{1}{2\pi\Gamma(K)} \oint_{C_\theta} \frac{\theta\Gamma(K)}{\theta^2 - x^2} d\theta = \frac{2\pi i}{2\Gamma(K)} \cdot \Gamma(K) \cdot 1 = i \tag{4.21}$$

For the second integral involving the incomplete gamma function:

$$\frac{1}{2\pi\Gamma(K)} \oint_{C_\theta} \frac{\theta e^{i\theta}\Gamma(K, i\theta)}{\theta^2 - x^2} d\theta \tag{4.22}$$

This can be evaluated by recognizing that for $\theta = x$:

$$\text{Res}_{\theta=x} \frac{\theta e^{i\theta}\Gamma(K, i\theta)}{\theta^2 - x^2} = \frac{x e^{ix}\Gamma(K, ix)}{2x} = \frac{e^{ix}\Gamma(K, ix)}{2}$$

And for $\theta = -x$:

$$\text{Res}_{\theta=-x} \frac{\theta e^{i\theta}\Gamma(K, i\theta)}{\theta^2 - x^2} = \frac{-x e^{-ix}\Gamma(K, -ix)}{-2x} = \frac{e^{-ix}\Gamma(K, -ix)}{2}$$

The exact solution to this part of the integral split is:

$$i + \frac{1}{\Gamma(K)} \left[\frac{ae^i x + be^i x}{2i}\right]$$
$$a := \Gamma(k, ix)$$
$$b := \Gamma(k, -ix)$$

(??) Where similarity to the complex exponential form of $cos(x)$ is noted.
This can be simplified using the properties of complex conjugation(??) to:

$$i \left[1 - \frac{\Re(e^{ix}\Gamma(K, ix))}{\Gamma(K)}\right]$$

Hartog's theorem[] further reduces the expression $\Re(e^{ix}\Gamma(K,ix))$ to:

$$\Re\left(\omega^K \sum_{j=0}^{\infty} \frac{\omega^j\,\Gamma(K)}{\Gamma(K+j+1)}\right)$$
$$= \Re\left(\frac{\omega^K}{K} \sum_{j=1}^{\infty} \frac{\omega^j}{\Gamma(K+j)}\right)$$
$$= \Re\left(\frac{\omega^K}{\Gamma(K+1)} \sum_{j=1}^{\infty} \frac{\omega^j}{K^{(j)}}\right)$$

Where $K^{(j)}$ is the rising factorial.

Noticing that the outside term $\frac{\omega^K}{\Gamma(K+1)}$ will always be real-valued because $K \in \mathbb{N}$, $K \geq 1$, $N \geq 1$ & $K = M + 2 = 2N + 2 \implies K$ is even) means it can be moved outside. We subsequently conclude, using the linearity of $\Re$ over a series sum, that $\iff$ j is even will a term be emitted. Thus:

$$= \omega^K \Gamma(K+1) \sum_{j=1}^{\infty} \frac{\omega^{2j}}{K^{(2j)}}$$
$$= \omega^K \Gamma(K+1) \sum_{j=0}^{\infty} \frac{(-1)^{j+1} x^{2j+2}}{K^{(2j+2)}}$$

This can be represented in terms of the hypogeometric series*[]. Substituting values we finally arrive at the answer:

$$\boxed{i\left(1 - \frac{x^2\,\omega^K}{K+1}\,\mathbb{H}\right)}$$
$$\mathbb{H} := {}_1F_2\left[\begin{matrix}1 & \frac{K}{2}+1 \\ & \frac{K}{2}+\frac{3}{2}\end{matrix}; \frac{-x^2}{4}\right]$$

Where for the special case $K = 0 \to \mathbb{H} = \mathrm{sinc}(x)$.

Approximation of the projector function

We can forgo an exact solution for an approximate solution using the mirror property[] of incomplete gamma functions & assuming $x > 0$, $x \in \mathbb{R}$. This is so that we can exploit the fact that the dominant contribution comes from the positive pole:

$$\frac{1}{2\pi\Gamma(K)} \oint_{C_\theta} \frac{\theta e^{i\theta}\Gamma(K,i\theta)}{\theta^2 - x^2} d\theta \approx \frac{2\pi i}{2\pi\Gamma(K)} \cdot \frac{e^{ix}\Gamma(K,ix)}{2} = \frac{ie^{ix}\Gamma(K,ix)}{2\Gamma(K)} \tag{4.23}$$

Combining both terms from (**??**):

$$\frac{1}{2\pi\Gamma(K)} \oint_{C_\theta} \left(\Gamma(K) - e^{i\theta}\Gamma(K,i\theta)\right)\left(\frac{\theta}{\theta^2 - x^2}\right) d\theta$$
$$= i - \frac{ie^{ix}\Gamma(K,ix)}{2\Gamma(K)}$$

Simplifying:

$$= i\left(1 - \frac{e^{ix}\Gamma(K, ix)}{2\Gamma(K)}\right)$$

This can be rewritten in terms of the regularized incomplete gamma function $Q(K, ix) = \Gamma(K, ix)/\Gamma(K)$:

$$\boxed{= i\left(1 - \frac{e^{ix}Q(K, ix)}{2}\right)} \tag{4.24}$$

Chapter 5

Control theoretic model for dynamic FPGA resource balancing

Appendix A

Appendix